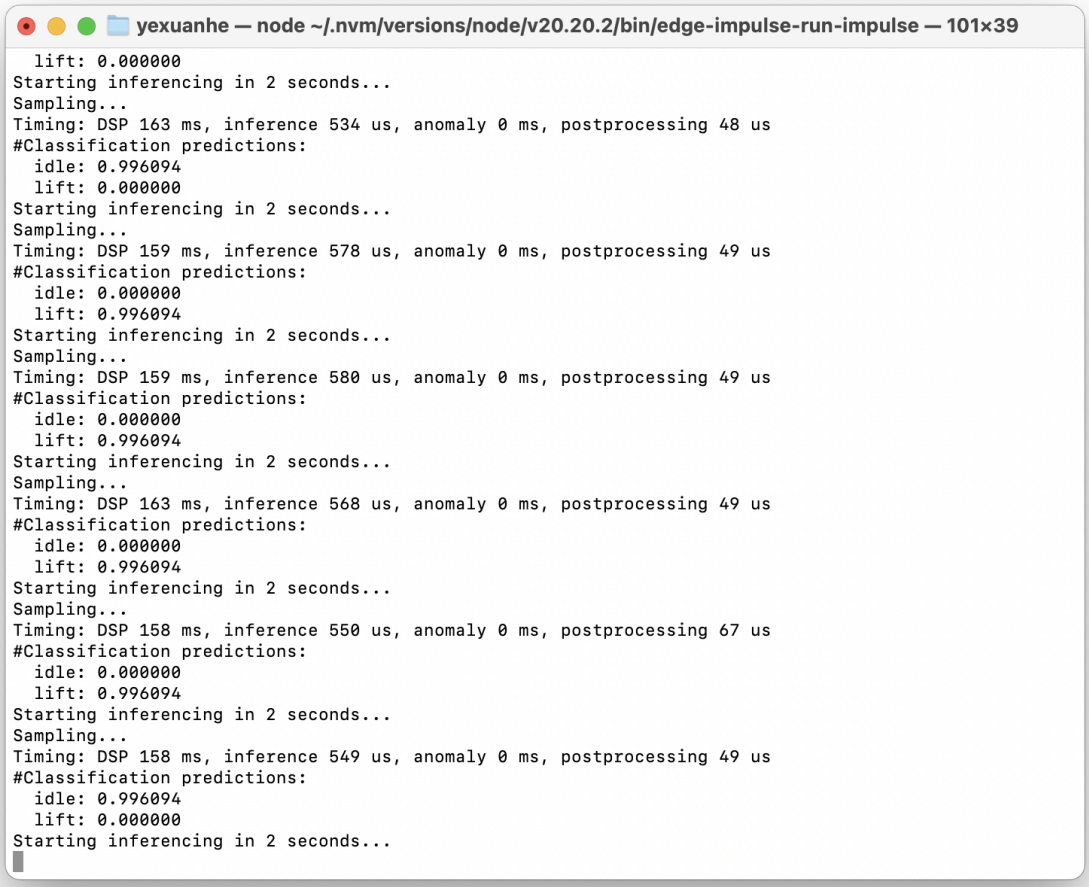
Part I

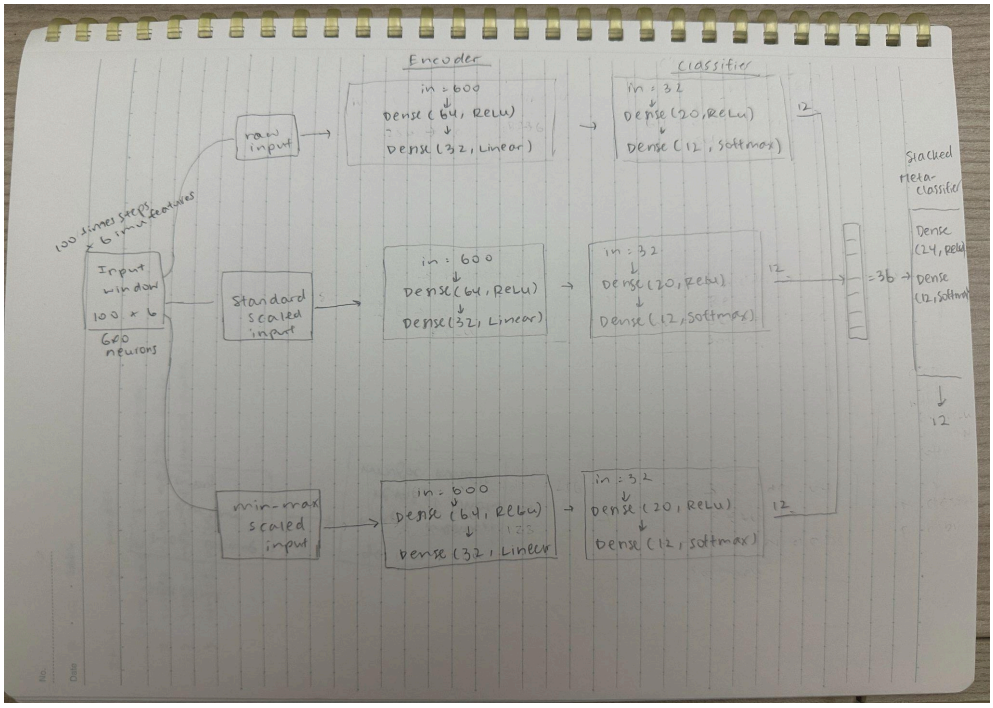
https://studio.edgeimpulse.com/public/1011587/live



Part II

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.



Question 2: Pruning and QAT methodology

How pruning is applied and what sparsity schedule is used

- The notebook applies magnitude-based pruning using prune_low_magnitude from the TensorFlow Model Optimization Toolkit (TF-MOT). It uses a PolynomialDecay sparsity schedule, starting at an initial sparsity of 10% (0.10) and gradually increasing to a final sparsity of 80% (0.80) over a defined number of steps (begin_step=0, end_step=500).

Why the pruning wrappers are stripped before QAT

- In order to convert the pruned model back into a standard Keras model (with the pruned weights physically set to zero). This is required because Keras/TF-MOT cannot easily stack both pruning wrappers and quantization wrappers on the same layer simultaneously without causing conflicts.

How QAT is applied after pruning

- After the pruning wrappers are stripped, Quantization-Aware Training (QAT) is applied using quantize_model within a quantize_scope. This wraps the standard Keras layers

with fake-quantization nodes that simulate the effects of lower precision (clamping and rounding errors) during an additional fine-tuning training phase.

Why the notebook uses an additional mask-enforcement step

- Since the pruning wrappers were stripped, the normal QAT fine-tuning optimizer would blindly update all weights—including the ones that were previously pruned to zero—effectively destroying the 80% sparsity. The notebook uses a custom MaskEnforcerCallback to manually re-apply the zero-masks to the weights at the end of every batch and epoch. This forces the pruned weights to remain exactly zero while the remaining active weights adapt to quantization.

How the final model is converted to an int8 TFLite model and why representative data is needed

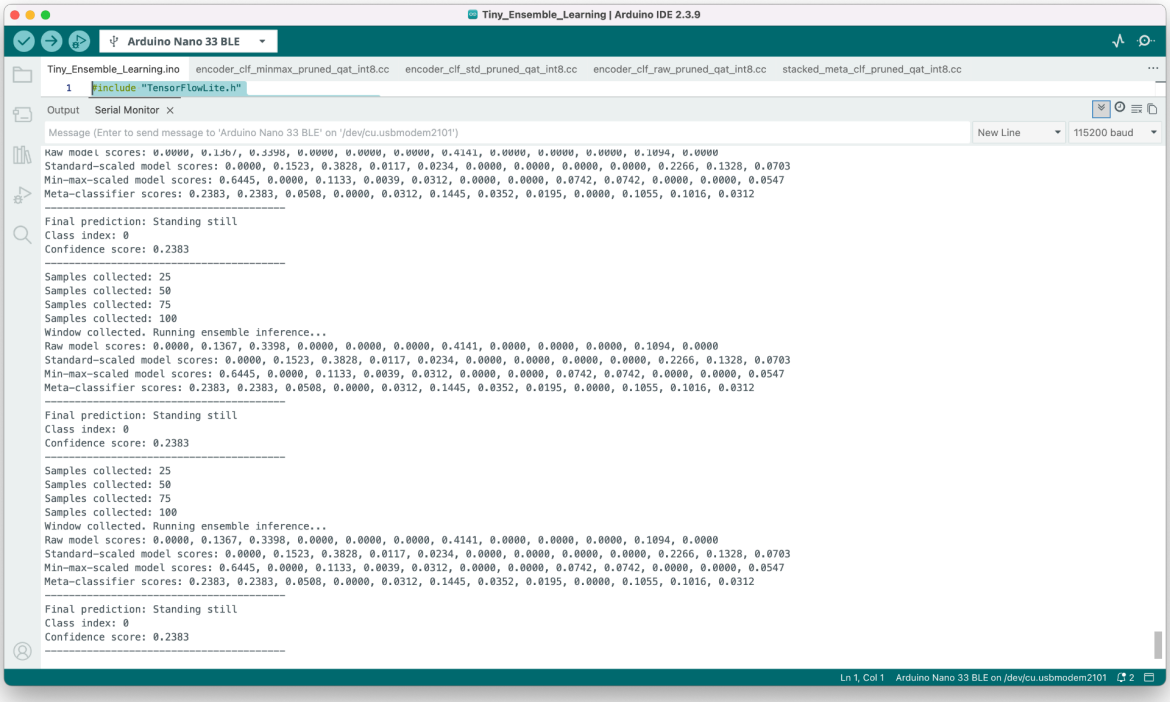
- The model is converted using tf.lite.TFLiteConverter with the target operations strictly set to TFLITE_BUILTINS_INT8. A representative dataset (a small subset of the training input windows) is required by the converter to pass realistic data through the model during conversion. This calibrates the dynamic ranges (min and max values) of the intermediate layer activations, allowing the converter to map the floating-point outputs safely into integer limits (-128 to 127) for full int8 inference.

How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense

- Int8 Quantization reduces the memory footprint of the weights by 4x (from 32-bit floats to 8-bit integers) and enables the use of faster, highly efficient integer arithmetic on the microcontroller's CPU.
- Pruning (80% sparsity) significantly increases the compressibility of the model payload, further reducing the Flash storage footprint. In some optimized TinyML engines, sparsity can also allow the CPU to skip zero-multiplications, saving computation cycles and battery power.

Question 3: Arduino deployment and real-world behavior

- A screenshot of the serial monitor output.



- A short explanation of whether the output is acceptable for the resting board condition.
    - Yes, the output is very good. It correctly and steadily shows that the board is not moving.
- Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.
    - It did briefly guess "Sitting and relaxing" once, but that is normal and acceptable. Because both "Sitting" and "Standing still" are resting states, they look very similar to the sensors (both just feel normal gravity).
- One or two practical changes that could improve deployment reliability.
    - We can use majority vote so the system looks at the last 5 to 10 guesses and picks the most common one to decide on. This ignores quick mistakes (like that single "Sitting" guess) and keeps the text on the screen from flickering.
    - We can also add a low-pass filter to the raw data before passing it into the neural network to block out tiny vibrations from the table or small electrical noise.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently? Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

- When I try to move the mcu up and down in a fast motion, the model gave all kinds of prediction including sitting, running, climbing stairs. Not so accurate.

- This might happen first: the way I hold the board in my hand is very different from how a sensor is strapped to a person's body in the real dataset. Second, the X, Y, and Z axes on the Arduino might be pointing in different directions than the original sensors. Also, the raw numbers coming from the Arduino might be in different units (like G-forces instead of m/s^2), so the math for the neural network gets confused because the scaling doesn't match the training data.